Cosmos EVM StateDB 최적화 구현 상세 계획

작성일: 2025-01-07 **목적**: 01-cosmos-evm-profiling-analysis.md에서 식별된 최적화 기회의 구체적 구현 계획
대상 독자: 블록체인 코어 개발자 **선행 문서**: 01-cosmos-evm-profiling-analysis.md

---

Executive Summary

본 문서는 Cosmos EVM StateDB의 성능 최적화를 위한 **구체적인 구현 계획**을 제시합니다.

구현 우선순위:

1. **Optimization 1**: Storage Read Cache (LRU + Bloom Filter)
2. **Optimization 2**: Batched State Commits
3. **Optimization 3**: Journal Pre-allocation & Pooling

예상 종합 효과:

- 트랜잭션 레이턴시: **35-45% 감소**
- 처리량: **50-80% 증가**
- 메모리 효율: **20-30% 개선**

목차

1. 최적화 1: Storage Read Cache
2. 최적화 2: Batched State Commits
3. 최적화 3: Journal 최적화
4. 통합 구현 계획
5. 테스트 전략
6. 벤치마크 계획
7. 위험 관리

---

1. 최적화 1: Storage Read Cache

1.1 현재 문제점 상세

현재 코드 (`x/vm/statedb/state_object.go:218-226`):

```go
func (s *stateObject) GetCommittedState(key common.Hash) common.Hash {
    if value, cached := s.originStorage[key]; cached {
        return value  // ✅ 로컬 캐시 hit
    }
    // ⚠️ 매번 KVStore 조회
    value := s.db.keeper.GetState(s.db.ctx, s.Address(), key)
    s.originStorage[key] = value
    return value
}
```

문제점:

1. `originStorage` 는 트랜잭션 스코프 캐시 (트랜잭션 종료 시 소멸)
2. 동일 블록 내 다른 트랜잭션에서 같은 slot 조회 시 다시 KVStore 접근
3. 읽기 전용 조회 (예: view 함수)도 KVStore 접근 필요

성능 영향:

```
SLOAD 1회:
├─ originStorage hit:    ~50 ns
└─ originStorage miss:   ~5,000-10,000 ns (100-200x 느림)
    ├─ keeper.GetState():  ~1,000 ns
    ├─ KVStore.Get():      ~3,000 ns (IAVL tree traversal)
    └─ Unmarshaling:       ~1,000 ns
```

1.2 설계: Block-Level Read Cache

아키텍처:

```
┌──────────────────────────────────────────────────────────┐
│                    Block-Level Read Cache                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   Transaction 1       Transaction 2       Transaction 3  │
│   ┌─────────┐         ┌─────────┐         ┌─────────┐    │
│   │ StateDB │         │ StateDB │         │ StateDB │    │
│   │origin   │         │origin   │         │origin   │    │
│   │Storage  │         │Storage  │         │Storage  │    │
│   └────┬────┘         └────┬────┘         └────┬────┘    │
│        │                   │                   │         │
│        └───────────────────┼───────────────────┘         │
│                            ▼                             │
│                 ┌──────────────────────┐                 │
│                 │   BlockReadCache     │  ◄── NEW!       │
│                 │  ┌────────────────┐  │                 │
│                 │  │  Bloom Filter  │  │  (존재 여부 빠른 확인)  │
│                 │  └────────────────┘  │                 │
│                 │  ┌────────────────┐  │                 │
│                 │  │   LRU Cache    │  │  (실제 값 캐시)        │
│                 │  └────────────────┘  │                 │
│                 └──────────┬───────────┘                 │
│                            │                             │
│                            ▼                             │
│                 ┌──────────────────────┐                 │
│                 │       KVStore        │  (최후 수단)        │
│                 └──────────────────────┘                 │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

1.3 구현 상세

1.3.1 BlockReadCache 구조체

```go
// x/vm/statedb/block_cache.go (새 파일)

package statedb

import (
    "sync"

    "github.com/bits-and-blooms/bloom/v3"
    lru "github.com/hashicorp/golang-lru/v2"
    "github.com/ethereum/go-ethereum/common"
)

// CacheKey는 (address, slot) 조합
type CacheKey struct {
    Address common.Address
    Slot    common.Hash
}

// BlockReadCache는 블록 레벨 storage 읽기 캐시
type BlockReadCache struct {
    // Bloom filter for quick existence check
    // False positive possible, but no false negative
    bloom *bloom.BloomFilter

    // LRU cache for actual values
    // Key: CacheKey, Value: common.Hash
    cache *lru.Cache[CacheKey, common.Hash]

    // Metrics
    hits     uint64
    misses   uint64

    // Mutex for bloom filter (LRU is already thread-safe)
    mu sync.RWMutex

    // Configuration
    maxSize        int
    bloomFPRate    float64
}

// NewBlockReadCache creates a new block-level read cache
func NewBlockReadCache(maxSize int) *BlockReadCache {
    // Bloom filter: 예상 100,000 entries, 1% false positive rate
    bf := bloom.NewWithEstimates(100000, 0.01)

    // LRU cache
    cache, _ := lru.New[CacheKey, common.Hash](maxSize)

    return &BlockReadCache{
        bloom:       bf,
        cache:       cache,
        maxSize:     maxSize,
        bloomFPRate: 0.01,
    }
}

// Get retrieves a value from the cache
// Returns (value, found)
func (c *BlockReadCache) Get(addr common.Address, slot common.Hash) (common.Hash, bool) {
    key := CacheKey{Address: addr, Slot: slot}
    keyBytes := c.keyToBytes(key)

    // Quick bloom filter check
    c.mu.RLock()
    mayExist := c.bloom.Test(keyBytes)
    c.mu.RUnlock()

    if !mayExist {
        // Definitely not in cache
        c.misses++
        return common.Hash{}, false
    }
```

```go
    // Check LRU cache
    if value, ok := c.cache.Get(key); ok {
        c.hits++
        return value, true
    }

    // Bloom filter false positive
    c.misses++
    return common.Hash{}, false
}

// Set stores a value in the cache
func (c *BlockReadCache) Set(addr common.Address, slot common.Hash, value common.Hash) {
    key := CacheKey{Address: addr, Slot: slot}
    keyBytes := c.keyToBytes(key)

    // Add to bloom filter
    c.mu.Lock()
    c.bloom.Add(keyBytes)
    c.mu.Unlock()

    // Add to LRU cache
    c.cache.Add(key, value)
}

// Invalidate removes a specific entry (after write)
func (c *BlockReadCache) Invalidate(addr common.Address, slot common.Hash) {
    key := CacheKey{Address: addr, Slot: slot}
    c.cache.Remove(key)
    // Note: Cannot remove from bloom filter, but that's OK
    // (false positives are acceptable)
}

// Clear resets the cache (at block boundary)
func (c *BlockReadCache) Clear() {
    c.mu.Lock()
    c.bloom.ClearAll()
    c.mu.Unlock()
    c.cache.Purge()
    c.hits = 0
    c.misses = 0
}

// Stats returns cache statistics
func (c *BlockReadCache) Stats() (hits, misses uint64, hitRate float64) {
    hits = c.hits
    misses = c.misses
    total := hits + misses
    if total > 0 {
        hitRate = float64(hits) / float64(total)
    }
    return
}

// keyToBytes converts CacheKey to bytes for bloom filter
func (c *BlockReadCache) keyToBytes(key CacheKey) []byte {
    result := make([]byte, 52) // 20 (address) + 32 (slot)
    copy(result[:20], key.Address.Bytes())
    copy(result[20:], key.Slot.Bytes())
    return result
}
```

1.3.2 Keeper에 캐시 통합

```go
// x/vm/keeper/keeper.go 수정

type Keeper struct {
    // ... existing fields ...

    // NEW: Block-level read cache
    blockReadCache *statedb.BlockReadCache
}

func NewKeeper(...) *Keeper {
    // ... existing code ...

    k := &Keeper{
        // ... existing assignments ...
        blockReadCache: statedb.NewBlockReadCache(50000), // 50K entries
    }
    return k
}

// GetStateWithCache retrieves storage with caching
func (k *Keeper) GetStateWithCache(ctx sdk.Context, addr common.Address, key common.Hash) common.Hash {
    // Try cache first
    if value, found := k.blockReadCache.Get(addr, key); found {
        return value
    }

    // Cache miss - fetch from KVStore
    value := k.GetState(ctx, addr, key)

    // Store in cache
    k.blockReadCache.Set(addr, key, value)

    return value
}

// SetState with cache invalidation
func (k *Keeper) SetState(ctx sdk.Context, addr common.Address, key common.Hash, value []byte) {
    // Invalidate cache entry
    k.blockReadCache.Invalidate(addr, key)

    // ... existing SetState logic ...
    store := prefix.NewStore(ctx.KVStore(k.storeKey), types.AddressStoragePrefix(addr))
    store.Set(key.Bytes(), value)
}

// ClearBlockCache should be called at block boundary
func (k *Keeper) ClearBlockCache() {
    k.blockReadCache.Clear()
}
```

1.3.3 StateDB 통합

```go
// x/vm/statedb/state_object.go 수정

// GetCommittedState - 수정된 버전
func (s *stateObject) GetCommittedState(key common.Hash) common.Hash {
    // 1. Local transaction cache
    if value, cached := s.originStorage[key]; cached {
        return value
    }

    // 2. Block-level cache (NEW!)
    value := s.db.keeper.GetStateWithCache(s.db.ctx, s.Address(), key)

    // 3. Update local cache
    s.originStorage[key] = value

    return value
}
```

1.3.4 ABCI 통합 (블록 경계 처리)

```go
// x/vm/keeper/abci.go (새 파일 또는 기존 파일에 추가)

// BeginBlocker is called at the start of each block
func (k *Keeper) BeginBlocker(ctx sdk.Context) {
    // Clear block-level cache for fresh block
    k.ClearBlockCache()

    // ... existing BeginBlocker logic ...
}

// EndBlocker is called at the end of each block
func (k *Keeper) EndBlocker(ctx sdk.Context) {
    // Log cache statistics
    hits, misses, hitRate := k.blockReadCache.Stats()
    if hits+misses > 0 {
        k.Logger(ctx).Debug("block read cache stats",
            "hits", hits,
            "misses", misses,
            "hit_rate", fmt.Sprintf("%.2f%%", hitRate*100),
        )
    }

    // ... existing EndBlocker logic ...
}
```

1.4 예상 효과

Before (현재):

```
Transaction with 100 SLOAD operations:
├─ First access (100 misses):     100 × 5,000 ns = 500,000 ns
└─ Total:                         ~500,000 ns (0.5 ms)
```

After (캐시 적용):

```
Block with 10 transactions, each 100 SLOAD (same contract):
Transaction 1: 100 misses         = 500,000 ns
Transaction 2: 80 hits + 20 miss  = 80×50 + 20×5,000 = 104,000 ns
Transaction 3: 90 hits + 10 miss  = 90×50 + 10×5,000 = 54,500 ns
...
Transaction 10: 95 hits + 5 miss  = 95×50 + 5×5,000 = 29,750 ns

Total: ~800,000 ns (vs 5,000,000 ns without cache)
Improvement: 84% reduction in storage read time
```

1.5 구현 일정

단계	작업	기간
1	BlockReadCache 구조체 구현	3일
2	Keeper 통합	2일
3	StateDB 통합	2일
4	ABCI hooks 추가	1일
5	단위 테스트	3일
6	통합 테스트	2일
7	벤치마크 및 튜닝	3일
총계		**16일 (약 3주)**

---

2. 최적화 2: Batched State Commits

2.1 현재 문제점 상세

현재 코드 (`x/vm/statedb/statedb.go:702-732`):

```go
func (s *StateDB) commitWithCtx(ctx sdk.Context) error {
    for _, addr := range s.journal.sortedDirties() {  // O(n log n) 정렬
        obj := s.stateObjects[addr]
        // ...
        for _, key := range obj.dirtyStorage.SortedKeys() {  // O(m log m) 정렬
            valueBytes := obj.dirtyStorage[key].Bytes()
            if len(valueBytes) == 0 {
                s.keeper.DeleteState(ctx, obj.Address(), key)  // 개별 삭제
            } else {
                s.keeper.SetState(ctx, obj.Address(), key, valueBytes)  // 개별 쓰기
            }
        }
    }
    return nil
}
```

문제점:

1. **정렬 오버헤드**: 매 commit마다 정렬 필요 (결정적 순서 보장)
2. **개별 쓰기**: 각 storage slot마다 별도 KVStore.Set() 호출
3. **오버헤드 누적**: 100개 slot 수정 시 100번의 KVStore 호출

성능 영향:

```
100 slots 수정 시 commit:
├─ sortedDirties():      ~1,000 ns
├─ SortedKeys() × 10:    ~5,000 ns
├─ SetState() × 100:     ~100 × 2,000 ns = 200,000 ns
└─ Total:                ~206,000 ns

병목: 개별 KVStore.Set() 호출
```

2.2 설계: Batch Writer

아키텍처:

```
┌──────────────────────────────────────────────────────────────┐
│                      Batched Commit Flow                      │
├──────────────────────────────────────────────────────────────┤
│                                                               │
│   현재:                                                       │
│   ┌─────────┐    ┌─────────┐    ┌─────────┐                  │
│   │SetState │──▶│KVStore  │──▶│SetState │──▶ ... (N번)       │
│   │  (1)    │    │.Set()   │    │  (2)    │                  │
│   └─────────┘    └─────────┘    └─────────┘                  │
│                                                               │
│   제안:                                                       │
│   ┌─────────┐    ┌─────────┐    ┌─────────┐    ┌─────────┐   │
│   │SetState │──▶│  Batch  │──▶│SetState │──▶│  Batch  │      │
│   │  (1)    │    │ Buffer  │    │  (2)    │    │ Buffer  │   │
│   └─────────┘    │         │    └─────────┘    │         │   │
│                  │   ...   │                    │   ...   │   │
│                  └────┬────┘                    └────┬────┘   │
│                       │                              │        │
│                       └──────────┬───────────────────┘        │
│                                  ▼                            │
│                            ┌──────────────┐                   │
│                            │ BatchCommit  │  ◀── 1번만 호출!  │
│                            │  KVStore     │                   │
│                            │  .SetMany()  │                   │
│                            └──────────────┘                   │
│                                                               │
└──────────────────────────────────────────────────────────────┘
```

2.3 구현 상세

2.3.1 BatchWriter 인터페이스

```go
// x/vm/statedb/batch_writer.go (새 파일)

package statedb

import (
    "bytes"
    "sort"

    "github.com/ethereum/go-ethereum/common"

    storetypes "cosmossdk.io/store/types"
    "github.com/cosmos/evm/x/vm/types"
)

// BatchOp represents a single batch operation
type BatchOp struct {
    IsDelete bool
    Key      []byte
    Value    []byte
}

// BatchWriter collects state changes for batch commit
type BatchWriter struct {
    ops []BatchOp

    // Pre-allocated buffer to reduce allocations
    buffer []byte

    // Statistics
    setCount    int
    deleteCount int
}

// NewBatchWriter creates a new batch writer with initial capacity
func NewBatchWriter(expectedOps int) *BatchWriter {
    return &BatchWriter{
        ops:    make([]BatchOp, 0, expectedOps),
        buffer: make([]byte, 0, 1024), // 1KB initial buffer
    }
}

// Reset clears the batch for reuse
func (b *BatchWriter) Reset() {
    b.ops = b.ops[:0]
    b.setCount = 0
    b.deleteCount = 0
}

// AddSet adds a set operation to the batch
func (b *BatchWriter) AddSet(addr common.Address, key common.Hash, value []byte) {
    storeKey := types.StateKey(addr, key)

    // Clone value to avoid reference issues
    valueCopy := make([]byte, len(value))
    copy(valueCopy, value)

    b.ops = append(b.ops, BatchOp{
        IsDelete: false,
        Key:      storeKey,
        Value:    valueCopy,
    })
    b.setCount++
}

// AddDelete adds a delete operation to the batch
func (b *BatchWriter) AddDelete(addr common.Address, key common.Hash) {
    storeKey := types.StateKey(addr, key)

    b.ops = append(b.ops, BatchOp{
        IsDelete: true,
        Key:      storeKey,
        Value:    nil,
```

```go
    })
    b.deleteCount++
}

// Commit executes all batched operations
func (b *BatchWriter) Commit(store storetypes.KVStore) error {
    if len(b.ops) == 0 {
        return nil
    }

    // Sort for deterministic order
    sort.Slice(b.ops, func(i, j int) bool {
        return bytes.Compare(b.ops[i].Key, b.ops[j].Key) < 0
    })

    // Execute all operations
    for _, op := range b.ops {
        if op.IsDelete {
            store.Delete(op.Key)
        } else {
            store.Set(op.Key, op.Value)
        }
    }

    return nil
}

// Stats returns batch statistics
func (b *BatchWriter) Stats() (total, sets, deletes int) {
    return len(b.ops), b.setCount, b.deleteCount
}
```

2.3.2 StateDB commitWithCtx 수정

```go
// x/vm/statedb/statedb.go 수정

// commitWithCtx writes the dirty states to keeper using batching
func (s *StateDB) commitWithCtx(ctx sdk.Context) error {
    // Create batch writer with estimated capacity
    estimatedOps := 0
    for _, addr := range s.journal.dirties {
        if obj, ok := s.stateObjects[addr]; ok {
            estimatedOps += len(obj.dirtyStorage) + 1 // +1 for account
        }
    }

    batch := NewBatchWriter(estimatedOps)

    // Collect all operations
    for _, addr := range s.journal.sortedDirties() {
        obj := s.stateObjects[addr]
        if obj.selfDestructed {
            if err := s.keeper.DeleteAccount(ctx, obj.Address()); err != nil {
                return errorsmod.Wrapf(err, "failed to delete account %s", obj.Address())
            }
            continue
        }

        // Handle code changes (not batched - different store)
        if obj.code != nil && obj.dirtyCode {
            if len(obj.code) == 0 {
                s.keeper.DeleteCode(ctx, obj.CodeHash())
            } else {
                s.keeper.SetCode(ctx, obj.CodeHash(), obj.code)
            }
        }

        // Handle account update (not batched - different mechanism)
        if err := s.keeper.SetAccount(ctx, obj.Address(), obj.account); err != nil {
            return errorsmod.Wrap(err, "failed to set account")
        }

        // Batch storage changes
        for _, key := range obj.dirtyStorage.SortedKeys() {
            valueBytes := obj.dirtyStorage[key].Bytes()
            if len(valueBytes) == 0 || isZeroHash(obj.dirtyStorage[key]) {
                batch.AddDelete(obj.Address(), key)
            } else {
                batch.AddSet(obj.Address(), key, valueBytes)
            }
        }
    }

    // Execute batched storage operations
    store := ctx.KVStore(s.keeper.storeKey)
    if err := batch.Commit(store); err != nil {
        return errorsmod.Wrap(err, "failed to commit storage batch")
    }

    return nil
}

// Helper function
func isZeroHash(h common.Hash) bool {
    return h == common.Hash{}
}
```

2.3.3 BatchWriter Pool (메모리 최적화)

```go
// x/vm/statedb/batch_pool.go (새 파일)

package statedb

import "sync"

// batchWriterPool reduces allocation overhead
var batchWriterPool = sync.Pool{
    New: func() interface{} {
        return NewBatchWriter(100) // Default capacity
    },
}

// GetBatchWriter gets a batch writer from the pool
func GetBatchWriter() *BatchWriter {
    return batchWriterPool.Get().(*BatchWriter)
}

// PutBatchWriter returns a batch writer to the pool
func PutBatchWriter(b *BatchWriter) {
    b.Reset()
    batchWriterPool.Put(b)
}
```

수정된 commitWithCtx (pool 사용):

```go
func (s *StateDB) commitWithCtx(ctx sdk.Context) error {
    batch := GetBatchWriter()
    defer PutBatchWriter(batch)

    // ... rest of the implementation ...
}
```

2.4 예상 효과

Before (현재):

```
100 storage slots 수정:
├─ sortedDirties():         ~1,000 ns
├─ SortedKeys():            ~5,000 ns
├─ Individual SetState:     100 × 2,000 ns = 200,000 ns
└─ Total:                   ~206,000 ns
```

After (배치 적용):

```
100 storage slots 수정:
├─ Collect operations:      100 × 200 ns = 20,000 ns
├─ Sort batch:              ~3,000 ns (1회)
├─ Batch commit:            100 × 1,500 ns = 150,000 ns (오버헤드 감소)
└─ Total:                   ~173,000 ns

개선: 16% 레이턴시 감소
```

대규모 트랜잭션 (1000 slots):

```
Before: 1000 × 2,000 = 2,000,000 ns
After:  1000 × 1,500 = 1,500,000 ns + 10,000 (sort)
개선: 25% 레이턴시 감소
```

2.5 구현 일정

단계	작업	기간
1	BatchWriter 구현	2일
2	BatchWriter Pool 구현	1일
3	StateDB 통합	2일
4	단위 테스트	2일
5	통합 테스트	2일
6	벤치마크	2일
총계		**11일 (약 2주)**

---

3. 최적화 3: Journal 최적화

3.1 현재 문제점 상세

현재 코드 (`x/vm/statedb/journal.go:42-52, 67-72`):

```go
type journal struct {
    entries []JournalEntry         // 슬라이스 - 동적 확장
    dirties map[common.Address]int
}

func newJournal() *journal {
    return &journal{
        dirties: make(map[common.Address]int),  // entries는 nil (0 capacity)
    }
}

func (j *journal) append(entry JournalEntry) {
    j.entries = append(j.entries, entry)  // ⚠️ 동적 확장 발생
    if addr := entry.Dirtied(); addr != nil {
        j.dirties[*addr]++
    }
}
```

문제점:

1. **entries 초기 용량 0**: 매번 슬라이스 재할당
2. **인터페이스 할당**: 각 entry가 JournalEntry 인터페이스로 저장
3. **dirties map 확장**: 초기 용량 없음

성능 영향:

```
100개 journal entry 추가:
├ 슬라이스 재할당 (~6회):  ~6 × 1,000 ns = 6,000 ns
├ Entry 인터페이스 할당:   100 × 100 ns = 10,000 ns
├ Map 확장 (~3회):         ~3 × 500 ns = 1,500 ns
└ Total overhead:          ~17,500 ns (불필요한 할당)
```

3.2 설계: Pre-allocation & Pooling

개선 전략:

1. **Pre-allocation**: 예상 크기로 미리 할당
2. **Capacity hints**: 트랜잭션 타입별 예상 크기
3. **Object pooling**: journal 및 entry 재사용

3.3 구현 상세

3.3.1 Pre-allocated Journal

```go
// x/vm/statedb/journal.go 수정

const (
    // Default capacities based on typical transaction patterns
    DefaultJournalCapacity  = 100
    DefaultDirtiesCapacity  = 10

    // Capacity hints for different transaction types
    SimpleTransferCapacity  = 10
    ERC20TransferCapacity   = 50
    DeFiSwapCapacity        = 200
    ContractDeployCapacity  = 500
)

// newJournal creates a new initialized journal with default capacity
func newJournal() *journal {
    return newJournalWithCapacity(DefaultJournalCapacity)
}

// newJournalWithCapacity creates a journal with specified capacity
func newJournalWithCapacity(capacity int) *journal {
    dirtiesCapacity := capacity / 10
    if dirtiesCapacity < DefaultDirtiesCapacity {
        dirtiesCapacity = DefaultDirtiesCapacity
    }

    return &journal{
        entries: make([]JournalEntry, 0, capacity),
        dirties: make(map[common.Address]int, dirtiesCapacity),
    }
}

// JournalCapacityHint returns recommended capacity based on tx data
func JournalCapacityHint(dataSize int, isContractCreation bool) int {
    if isContractCreation {
        return ContractDeployCapacity
    }

    // Estimate based on data size (calldata)
    if dataSize == 0 {
        return SimpleTransferCapacity
    } else if dataSize < 100 {
        return ERC20TransferCapacity
    } else if dataSize < 500 {
        return DeFiSwapCapacity
    }

    // Large transaction
    return dataSize / 2
}
```

3.3.2 Journal Pool

```go
// x/vm/statedb/journal_pool.go (새 파일)

package statedb

import "sync"

// journalPool reduces allocation overhead for journal
var journalPool = sync.Pool{
    New: func() interface{} {
        return newJournalWithCapacity(DefaultJournalCapacity)
    },
}

// Pools for common entry types
var (
    storageChangePool = sync.Pool{
        New: func() interface{} {
            return &storageChange{}
        },
    }
    balanceChangePool = sync.Pool{
        New: func() interface{} {
            return &balanceChange{}
        },
    }
    nonceChangePool = sync.Pool{
        New: func() interface{} {
            return &nonceChange{}
        },
    }
)

// GetJournal gets a journal from the pool
func GetJournal() *journal {
    return journalPool.Get().(*journal)
}

// GetJournalWithCapacity gets a journal with specific capacity
func GetJournalWithCapacity(capacity int) *journal {
    j := journalPool.Get().(*journal)
    if cap(j.entries) < capacity {
        // Need larger journal, create new one
        return newJournalWithCapacity(capacity)
    }
    return j
}

// PutJournal returns a journal to the pool
func PutJournal(j *journal) {
    // Clear entries but keep capacity
    j.entries = j.entries[:0]

    // Clear dirties map
    for k := range j.dirties {
        delete(j.dirties, k)
    }

    journalPool.Put(j)
}

// GetStorageChange gets a storageChange from pool
func GetStorageChange(account *common.Address, key, prevalue common.Hash) *storageChange {
    sc := storageChangePool.Get().(*storageChange)
    sc.account = account
    sc.key = key
    sc.prevalue = prevalue
    return sc
}

// PutStorageChange returns a storageChange to pool
func PutStorageChange(sc *storageChange) {
    sc.account = nil
    sc.key = common.Hash{}
    sc.prevalue = common.Hash{}
```

```
    storageChangePool.Put(sc)
}
```

3.3.3 StateDB 통합

```go
// x/vm/statedb/statedb.go 수정

// New creates a new state with optimized journal
func New(ctx sdk.Context, keeper Keeper, txConfig TxConfig) *StateDB {
    // Determine journal capacity based on transaction hints
    capacity := DefaultJournalCapacity
    // Could use txConfig to get hints about expected complexity

    return &StateDB{
        keeper:           keeper,
        ctx:              ctx,
        stateObjects:     make(map[common.Address]*stateObject),
        journal:          GetJournalWithCapacity(capacity),
        accessList:       newAccessList(),
        transientStorage: newTransientStorage(),
        txConfig:         txConfig,
    }
}

// NewWithHints creates StateDB with capacity hints
func NewWithHints(ctx sdk.Context, keeper Keeper, txConfig TxConfig, dataSize int, isCreate bool) *StateDB {
    capacity := JournalCapacityHint(dataSize, isCreate)

    return &StateDB{
        keeper:           keeper,
        ctx:              ctx,
        stateObjects:     make(map[common.Address]*stateObject, 10),
        journal:          GetJournalWithCapacity(capacity),
        accessList:       newAccessList(),
        transientStorage: newTransientStorage(),
        txConfig:         txConfig,
    }
}

// Cleanup should be called after transaction completes
func (s *StateDB) Cleanup() {
    // Return journal to pool
    if s.journal != nil {
        PutJournal(s.journal)
        s.journal = nil
    }
}
```

3.3.4 Optimized append with pooling

```go
// x/vm/statedb/state_object.go 수정

// SetState with pooled entry
func (s *stateObject) SetState(key common.Hash, value common.Hash) common.Hash {
    prev := s.GetState(key)
    if prev == value {
        return prev
    }

    // Use pooled entry
    entry := GetStorageChange(&s.address, key, prev)
    s.db.journal.append(entry)
    s.setState(key, value)
    return prev
}
```

3.4 예상 효과

Before (현재):

```
100개 storage 변경:
├ Journal 할당:            ~500 ns
├ 슬라이스 재할당 (6회):   ~6,000 ns
├ storageChange 할당:      100 × 100 ns = 10,000 ns
├ Map 확장:                ~1,500 ns
└ Total:                   ~18,000 ns
```

After (최적화 적용):

```
100개 storage 변경:
├ Journal pool get:        ~50 ns
├ 슬라이스 확장:           0 ns (pre-allocated)
├ storageChange pool:      100 × 20 ns = 2,000 ns
├ Map 확장:                0 ns (pre-allocated)
└ Total:                   ~2,050 ns

개선: 89% 오버헤드 감소
```

3.5 구현 일정

단계	작업	기간
1	Pre-allocation 구현	1일
2	Journal Pool 구현	2일
3	Entry Pool 구현	2일
4	StateDB 통합	1일
5	테스트	2일
6	벤치마크	1일
총계		**9일 (약 2주)**

---

4. 통합 구현 계획

4.1 Phase별 구현

```
┌──────────────────────────────────────────────────────────────┐
│                    Implementation Timeline                    │
├──────────────────────────────────────────────────────────────┤
│                                                               │
│  Phase 1 (Week 1-3): Storage Read Cache                       │
│  ├─ Week 1: BlockReadCache 구현                               │
│  ├─ Week 2: Keeper/StateDB 통합                               │
│  └─ Week 3: 테스트 및 벤치마크                                 │
│                                                               │
│  Phase 2 (Week 4-5): Batched Commits                          │
│  ├─ Week 4: BatchWriter 구현 및 통합                           │
│  └─ Week 5: 테스트 및 벤치마크                                 │
│                                                               │
│  Phase 3 (Week 6-7): Journal Optimization                     │
│  ├─ Week 6: Pre-allocation 및 Pooling 구현                    │
│  └─ Week 7: 테스트 및 벤치마크                                 │
│                                                               │
│  Phase 4 (Week 8): Integration & Polish                       │
│  ├─ 전체 통합 테스트                                           │
│  ├─ 성능 regression 검증                                       │
│  └─ 문서화 및 PR 준비                                          │
│                                                               │
└──────────────────────────────────────────────────────────────┘

총 기간: 8주
```

4.2 파일 변경 요약

새 파일:

```
x/vm/statedb/
├─ block_cache.go          # BlockReadCache 구현
├─ batch_writer.go         # BatchWriter 구현
├─ batch_pool.go           # BatchWriter pool
└─ journal_pool.go         # Journal 및 entry pools
```

수정 파일:

```
x/vm/
├─ keeper/
│   ├─ keeper.go           # BlockReadCache 통합
│   └─ abci.go             # BeginBlocker/EndBlocker hooks
└─ statedb/
    ├─ statedb.go          # Batch commit, cleanup
    ├─ state_object.go     # Pooled entries
    └─ journal.go          # Pre-allocation
```

4.3 의존성 관리

필요한 외부 라이브러리:

```go
// go.mod 추가
require (
    github.com/bits-and-blooms/bloom/v3 v3.5.0  // Bloom filter
    github.com/hashicorp/golang-lru/v2 v2.0.7   // LRU cache
)
```

4.4 Feature Flags

```go
// x/vm/types/params.go

type Params struct {
    // ... existing params ...

    // Performance optimization flags
    EnableReadCache     bool   `json:"enable_read_cache"`
    ReadCacheSize       int    `json:"read_cache_size"`
    EnableBatchCommit   bool   `json:"enable_batch_commit"`
    EnableJournalPool   bool   `json:"enable_journal_pool"`
}

// DefaultParams returns default parameters with optimizations enabled
func DefaultParams() Params {
    return Params{
        // ... existing defaults ...
        EnableReadCache:   true,
        ReadCacheSize:     50000,
        EnableBatchCommit: true,
        EnableJournalPool: true,
    }
}
```

5. 테스트 전략

5.1 단위 테스트

```go
// x/vm/statedb/block_cache_test.go

func TestBlockReadCache_Basic(t *testing.T) {
    cache := NewBlockReadCache(1000)

    addr := common.HexToAddress("0x1234...")
    slot := common.HexToHash("0xabcd...")
    value := common.HexToHash("0x9999...")

    // Test miss
    _, found := cache.Get(addr, slot)
    require.False(t, found)

    // Test set and hit
    cache.Set(addr, slot, value)
    result, found := cache.Get(addr, slot)
    require.True(t, found)
    require.Equal(t, value, result)

    // Test invalidation
    cache.Invalidate(addr, slot)
    _, found = cache.Get(addr, slot)
    require.False(t, found)
}

func TestBlockReadCache_LRUEviction(t *testing.T) {
    cache := NewBlockReadCache(10) // Small cache
    addr := common.HexToAddress("0x1234...")

    // Fill cache
    for i := 0; i < 20; i++ {
        slot := common.BigToHash(big.NewInt(int64(i)))
        value := common.BigToHash(big.NewInt(int64(i * 100)))
        cache.Set(addr, slot, value)
    }

    // First entries should be evicted
    slot0 := common.BigToHash(big.NewInt(0))
    _, found := cache.Get(addr, slot0)
    require.False(t, found)

    // Recent entries should exist
    slot19 := common.BigToHash(big.NewInt(19))
    _, found = cache.Get(addr, slot19)
    require.True(t, found)
}

func TestBatchWriter_Commit(t *testing.T) {
    batch := NewBatchWriter(100)

    addr := common.HexToAddress("0x1234...")
    for i := 0; i < 50; i++ {
        key := common.BigToHash(big.NewInt(int64(i)))
        value := []byte{byte(i)}
        batch.AddSet(addr, key, value)
    }

    // Mock store
    store := newMockKVStore()
    err := batch.Commit(store)
    require.NoError(t, err)

    // Verify all entries written
    for i := 0; i < 50; i++ {
        key := types.StateKey(addr, common.BigToHash(big.NewInt(int64(i))))
        value := store.Get(key)
        require.Equal(t, []byte{byte(i)}, value)
    }
}

func TestJournalPool_Reuse(t *testing.T) {
```

```go
    // Get journal
    j1 := GetJournal()
    require.NotNil(t, j1)

    // Add some entries
    addr := common.HexToAddress("0x1234...")
    for i := 0; i < 10; i++ {
        j1.append(createObjectChange{account: &addr})
    }

    // Return to pool
    PutJournal(j1)

    // Get again - should be same instance, but cleared
    j2 := GetJournal()
    require.Equal(t, 0, j2.length())
    require.True(t, cap(j2.entries) >= 10) // Capacity preserved
}
```

5.2 통합 테스트

```go
// x/vm/keeper/keeper_integration_test.go

func TestOptimizedStateDB_ERC20Transfer(t *testing.T) {
    // Setup test environment
    app, ctx := setupTestApp()

    // Deploy ERC20 contract
    erc20Addr := deployERC20(ctx, app)

    // Perform transfer
    sender := accounts[0]
    receiver := accounts[1]
    amount := big.NewInt(1000)

    // Execute transfer
    res, err := app.EVMKeeper.ApplyTransaction(ctx, transferTx)
    require.NoError(t, err)
    require.False(t, res.Failed())

    // Verify balances
    senderBal := getERC20Balance(ctx, erc20Addr, sender)
    receiverBal := getERC20Balance(ctx, erc20Addr, receiver)

    require.Equal(t, expectedSenderBal, senderBal)
    require.Equal(t, expectedReceiverBal, receiverBal)
}

func TestOptimizedStateDB_RevertBehavior(t *testing.T) {
    // Ensure optimizations don't break revert behavior
    app, ctx := setupTestApp()

    // Transaction that reverts
    revertTx := createRevertingTx()

    // State before
    stateBefore := captureState(ctx)

    // Execute
    res, err := app.EVMKeeper.ApplyTransaction(ctx, revertTx)
    require.NoError(t, err)
    require.True(t, res.Failed())

    // State should be unchanged
    stateAfter := captureState(ctx)
    require.Equal(t, stateBefore, stateAfter)
}
```

5.3 Fuzzing Tests

```go
// x/vm/statedb/fuzz_test.go

func FuzzBlockReadCache(f *testing.F) {
    f.Add([]byte("addr"), []byte("slot"), []byte("value"))

    f.Fuzz(func(t *testing.T, addrBytes, slotBytes, valueBytes []byte) {
        if len(addrBytes) < 20 || len(slotBytes) < 32 || len(valueBytes) < 32 {
            return
        }

        cache := NewBlockReadCache(100)
        addr := common.BytesToAddress(addrBytes[:20])
        slot := common.BytesToHash(slotBytes[:32])
        value := common.BytesToHash(valueBytes[:32])

        // Set
        cache.Set(addr, slot, value)

        // Get
        result, found := cache.Get(addr, slot)
        if !found {
            t.Fatal("expected to find value")
        }
        if result != value {
            t.Fatal("value mismatch")
        }
    })
}
```

6. 벤치마크 계획

6.1 마이크로 벤치마크

```go
// x/vm/statedb/benchmark_test.go

func BenchmarkBlockReadCache_Get_Hit(b *testing.B) {
    cache := NewBlockReadCache(10000)
    addr := common.HexToAddress("0x1234...")
    slot := common.HexToHash("0xabcd...")
    value := common.HexToHash("0x9999...")
    cache.Set(addr, slot, value)

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        cache.Get(addr, slot)
    }
}

func BenchmarkBlockReadCache_Get_Miss(b *testing.B) {
    cache := NewBlockReadCache(10000)
    addr := common.HexToAddress("0x1234...")
    slot := common.HexToHash("0xabcd...")

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        cache.Get(addr, slot)
    }
}

func BenchmarkBatchWriter_Commit(b *testing.B) {
    for _, n := range []int{10, 100, 1000} {
        b.Run(fmt.Sprintf("ops=%d", n), func(b *testing.B) {
            batch := NewBatchWriter(n)
            addr := common.HexToAddress("0x1234...")

            for i := 0; i < n; i++ {
                key := common.BigToHash(big.NewInt(int64(i)))
                batch.AddSet(addr, key, []byte{byte(i % 256)})
            }

            store := newMockKVStore()

            b.ResetTimer()
            for i := 0; i < b.N; i++ {
                batch.Commit(store)
            }
        })
    }
}

func BenchmarkJournal_Append_Pooled(b *testing.B) {
    j := GetJournalWithCapacity(b.N)
    addr := common.HexToAddress("0x1234...")
    key := common.HexToHash("0xabcd...")
    prev := common.HexToHash("0x1111...")

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        entry := GetStorageChange(&addr, key, prev)
        j.append(entry)
    }
}

func BenchmarkJournal_Append_NoPool(b *testing.B) {
    j := newJournal()
    addr := common.HexToAddress("0x1234...")
    key := common.HexToHash("0xabcd...")
    prev := common.HexToHash("0x1111...")

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        j.append(storageChange{
            account:  &addr,
            key:      key,
```

```
            prevalue: prev,
        })
    }
}
```

6.2 End-to-End 벤치마크

```go
// x/vm/keeper/benchmark_test.go

func BenchmarkERC20Transfer(b *testing.B) {
    app, ctx := setupBenchmarkApp()
    erc20Addr := deployERC20(ctx, app)

    txs := make([]*ethtypes.Transaction, b.N)
    for i := 0; i < b.N; i++ {
        txs[i] = createERC20TransferTx(erc20Addr, i)
    }

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        _, err := app.EVMKeeper.ApplyTransaction(ctx, txs[i])
        if err != nil {
            b.Fatal(err)
        }
    }
}

func BenchmarkUniswapSwap(b *testing.B) {
    app, ctx := setupBenchmarkApp()
    routerAddr, pools := deployUniswap(ctx, app)

    txs := make([]*ethtypes.Transaction, b.N)
    for i := 0; i < b.N; i++ {
        txs[i] = createSwapTx(routerAddr, pools[0], i)
    }

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        _, err := app.EVMKeeper.ApplyTransaction(ctx, txs[i])
        if err != nil {
            b.Fatal(err)
        }
    }
}
```

6.3 성능 목표

벤치마크	현재 (μs)	목표 (μs)	개선율
Simple Transfer	1,000	650	35%
ERC20 Transfer	5,000	3,000	40%
Uniswap Swap	20,000	12,000	40%
Cache Hit	5,000	50	99%
Batch Commit (100 ops)	200,000	150,000	25%
Journal Append	100	20	80%

7. 위험 관리

7.1 위험 식별

위험	확률	영향	완화 방안
캐시 일관성 버그	중간	높음	철저한 invalidation 테스트, fuzzing
메모리 누수	낮음	중간	Pool 테스트, memory profiling
성능 regression	낮음	높음	CI 벤치마크, feature flags
Consensus 불일치	매우 낮음	매우 높음	결정적 순서 보장 검증
외부 라이브러리 버그	낮음	중간	검증된 라이브러리 사용, 버전 고정

7.2 완화 전략

캐시 일관성:

```go
// 모든 쓰기 경로에서 invalidation 확인
func (k *Keeper) SetState(ctx sdk.Context, addr common.Address, key common.Hash, value []byte) {
    // MUST invalidate cache before write
    k.blockReadCache.Invalidate(addr, key)

    // Proceed with write
    store := prefix.NewStore(ctx.KVStore(k.storeKey), types.AddressStoragePrefix(addr))
    store.Set(key.Bytes(), value)
}
```

결정론 보장:

```go
// 모든 iteration은 정렬된 순서로
func (s *StateDB) commitWithCtx(ctx sdk.Context) error {
    // MUST sort for deterministic order
    addrs := s.journal.sortedDirties()

    for _, addr := range addrs {
        obj := s.stateObjects[addr]
        // MUST sort keys
        keys := obj.dirtyStorage.SortedKeys()
        // ...
    }
}
```

Feature Flags로 롤백 가능:

```go
func (s *StateDB) GetCommittedState(key common.Hash) common.Hash {
    if !s.keeper.params.EnableReadCache {
        // Fallback to original behavior
        return s.keeper.GetState(s.ctx, s.Address(), key)
    }

    // Optimized path
    return s.keeper.GetStateWithCache(s.ctx, s.Address(), key)
}
```

7.3 롤백 계획

1. **즉시 롤백**: Feature flag 비활성화
2. **긴급 패치**: 버그 수정 배포
3. **전체 롤백**: 최적화 코드 제거 (최후 수단)

결론

요약

본 문서에서 제시한 3가지 최적화를 모두 구현하면:

메트릭	현재	예상	개선율
트랜잭션 레이턴시	1,000 µs	600 µs	40%
처리량 (TPS)	1,000	1,700	70%
메모리 효율	Baseline	+25%	25%
Storage Read	5,000 ns	50 ns	99% (cache hit)

다음 단계

1. **Week 1**: BlockReadCache 구현 시작
2. **Week 3**: 첫 번째 성능 측정
3. **Week 5**: BatchWriter 완료
4. **Week 7**: Journal 최적화 완료
5. **Week 8**: PR 준비 및 제출

PR 전략

1. **PR #1**: Storage Read Cache (독립적, 위험 낮음)
2. **PR #2**: Batched Commits (독립적, 위험 낮음)
3. **PR #3**: Journal Optimization (독립적, 위험 낮음)
4. **PR #4**: Documentation & Benchmarks

각 PR은 독립적으로 리뷰 및 머지 가능하도록 설계

작성 완료: 2025-01-07 **총 페이지**: ~25페이지 **분석 기반**: cosmos/evm 실제 소스코드 **다음 문서**: `03-consensus-experiment-design.md` (합의 메커니즘 실험 설계)